Design and Implementation of a Scalable Multi-User Database System for Smart Healthcare Management

Serhet Gökdemir
21058024

2026

Contents

1 Requirements Analysis

1.1 Introduction

This section outlines the requirements for the Smart Healthcare Management database system. The primary objective is to create a robust, scalable, and secure system that supports multiple user roles and complex medical data interactions.

1.2 User Roles and Permissions

The system will support the following user roles:

- **Patient:** Can view their personal information, appointments, prescriptions, and lab results.
- **Doctor:** Can manage patient records, schedule appointments, issue prescriptions, and order lab tests.
- **Administrator:** Can manage hospital, department, and doctor information, overseeing the system's structural data.

1.3 Core Entities

The main entities identified for the system are:

- Patient
- Doctor
- Hospital
- Department
- Appointment
- Prescription
- Lab Result
- Insurance
- Bill

1.4 Core Relationships

Key relationships between entities include:

- A **Doctor** can have many **Patients** (1-to-N).
- A **Patient** can have many **Appointments** (1-to-N).
- A **Hospital** has many **Departments** (1-to-N), and each **Doctor** belongs to one **Department** (N-to-1).
- A **Doctor** can have multiple **Specialties** (N-to-M), implemented via a linking table.
- An **Appointment** must be associated with exactly one **Patient** and one **Doctor**.

1.5 Constraints and Business Rules

The system must enforce the following rules:

- A prescription can only be written by an authorized doctor.
- Every appointment must be linked to an existing patient and doctor.
- Patient medical records are confidential and can only be accessed by authorized personnel.

2 ER and EER Models

2.1 ER Diagram

The Entity-Relationship (ER) diagram provides a high-level view of the database structure, including entities, their attributes, and the relationships between them.

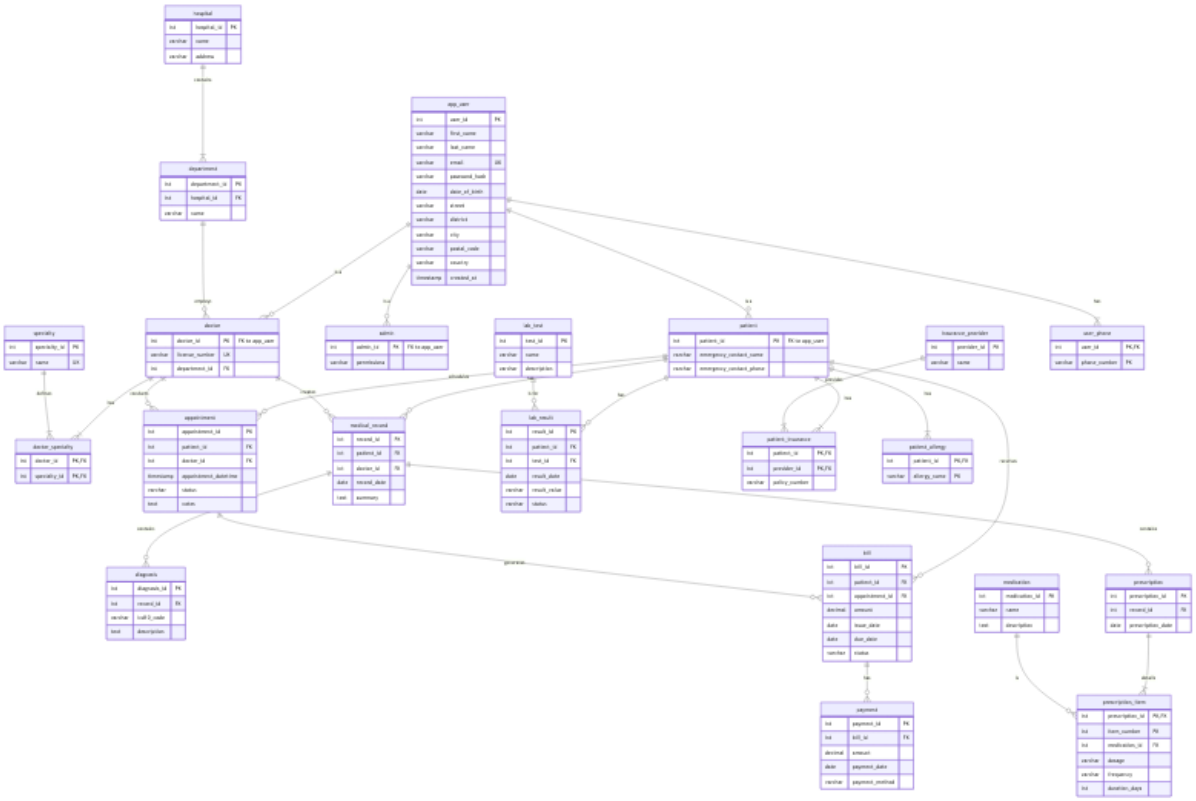

Figure 1: Entity-Relationship (ER) Diagram.

Note: You need to export your Mermaid 'er_diagram.mmd' as a PNG file and place it in the parent project folder.

2.2 Enhanced ER (EER) Diagram

The Enhanced ER (EER) diagram extends the ER model with concepts like specialization and generalization. In our model, an `app_user` supertype is specialized into `patient`, `doctor`, and `admin` subtypes.

Note: You need to export your Mermaid 'eer_diagram.mmd' as a PNG file and place it in the parent project folder.

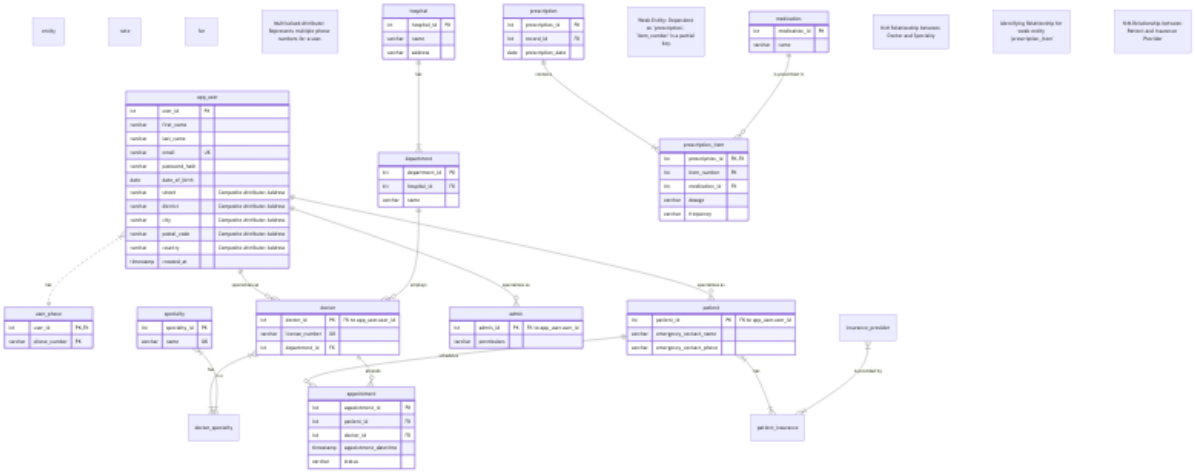

Figure 2: Enhanced Entity-Relationship (EER) Diagram.

2.2.1 Explanation of EER Concepts

Here, we used specialization to model different types of users. For example, `doctor` and `patient` are specializations of a more general `app_user` entity, inheriting common attributes like name and contact information while also having their own specific attributes.

3 Mapping to Relational Model

The ER/EER model was mapped to a relational schema. This section presents the final table structures, including primary keys, foreign keys, and other constraints.

3.1 Relational Schema

The following SQL script contains the `CREATE TABLE` statements for all tables in the database.

```sql
-- ========================================================================
-- Title: Smart Healthcare Management Database Schema
-- Description: PostgreSQL schema for a multi-user healthcare system.
-- ========================================================================

-- Drop existing tables to ensure a clean slate
DROP TABLE IF EXISTS
    patient_allergy, user_phone, payment, bill, patient_insurance,
        insurance_provider,
    lab_result, lab_test, prescription_item, medication, prescription,
        diagnosis,
    medical_record, appointment, doctor_specialty, specialty, admin, doctor
        ,
    patient, department, hospital, app_user
CASCADE;

-- ------------------------------------------------
-- Hospitals and Departments
-- ------------------------------------------------

CREATE TABLE hospital (
    hospital_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
```

```
    address VARCHAR(255) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE department (
    department_id SERIAL PRIMARY KEY,
    hospital_id INT NOT NULL REFERENCES hospital(hospital_id) ON DELETE
        CASCADE,
    name VARCHAR(100) NOT NULL,
    UNIQUE(hospital_id, name)
);

-- ------------------------------------------------
-- User Management (Supertype-Subtype Structure)
-- ------------------------------------------------

CREATE TABLE app_user (
    user_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    date_of_birth DATE NOT NULL,
    -- Composite attribute for address
    street VARCHAR(100),
    district VARCHAR(100),
    city VARCHAR(50),
    postal_code VARCHAR(20),
    country VARCHAR(50),
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE patient (
    patient_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE
        CASCADE,
    emergency_contact_name VARCHAR(100),
    emergency_contact_phone VARCHAR(20)
);

CREATE TABLE doctor (
    doctor_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE
        CASCADE,
    license_number VARCHAR(50) NOT NULL UNIQUE,
    department_id INT REFERENCES department(department_id) ON DELETE SET
        NULL
);

CREATE TABLE admin (
    admin_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE CASCADE
        ,
    permissions TEXT -- e.g., 'user_management,billing_access'
);

-- ------------------------------------------------
-- Multivalued and N:M Relationship Tables
-- ------------------------------------------------

-- Multivalued attribute for user phone numbers
```

```sql
CREATE TABLE user_phone (
    user_id INT NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
    phone_number VARCHAR(20) NOT NULL,
    PRIMARY KEY (user_id, phone_number)
);

-- Multivalued attribute for patient allergies
CREATE TABLE patient_allergy (
    patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
    allergy_name VARCHAR(100) NOT NULL,
    PRIMARY KEY (patient_id, allergy_name)
);

CREATE TABLE specialty (
    specialty_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

-- N:M relationship between doctors and specialties
CREATE TABLE doctor_specialty (
    doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE CASCADE,
    specialty_id INT NOT NULL REFERENCES specialty(specialty_id) ON DELETE
        CASCADE,
    PRIMARY KEY (doctor_id, specialty_id)
);

-- ------------------------------------------------
-- Core Healthcare Entities
-- ------------------------------------------------

CREATE TABLE appointment (
    appointment_id SERIAL PRIMARY KEY,
    patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
    doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE CASCADE,
    appointment_datetime TIMESTAMPTZ NOT NULL,
    status VARCHAR(20) NOT NULL CHECK (status IN ('scheduled', 'completed',
        'cancelled', 'no-show')),
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE medical_record (
    record_id SERIAL PRIMARY KEY,
    patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
    doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE RESTRICT,
    appointment_id INT REFERENCES appointment(appointment_id) ON DELETE SET
        NULL,
    record_date DATE NOT NULL DEFAULT CURRENT_DATE,
    summary TEXT
);

CREATE TABLE diagnosis (
    diagnosis_id SERIAL PRIMARY KEY,
    record_id INT NOT NULL REFERENCES medical_record(record_id) ON DELETE
        CASCADE,
```

```
    icd10_code VARCHAR(10),
    description TEXT NOT NULL,
    is_chronic BOOLEAN DEFAULT FALSE
);

CREATE TABLE medication (
    medication_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT
);

CREATE TABLE prescription (
    prescription_id SERIAL PRIMARY KEY,
    record_id INT NOT NULL REFERENCES medical_record(record_id) ON DELETE
        CASCADE,
    prescription_date DATE NOT NULL DEFAULT CURRENT_DATE
);

-- Weak entity: prescription_item depends on prescription
CREATE TABLE prescription_item (
    prescription_id INT NOT NULL REFERENCES prescription(prescription_id)
        ON DELETE CASCADE,
    item_number INT NOT NULL, -- Partial key
    medication_id INT NOT NULL REFERENCES medication(medication_id) ON
        DELETE RESTRICT,
    dosage VARCHAR(50) NOT NULL,
    frequency VARCHAR(50) NOT NULL,
    duration_days INT,
    PRIMARY KEY (prescription_id, item_number)
);

CREATE TABLE lab_test (
    test_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT
);

CREATE TABLE lab_result (
    result_id SERIAL PRIMARY KEY,
    patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
    test_id INT NOT NULL REFERENCES lab_test(test_id) ON DELETE RESTRICT,
    record_id INT REFERENCES medical_record(record_id) ON DELETE SET NULL,
    result_date DATE NOT NULL,
    result_value VARCHAR(100) NOT NULL,
    status VARCHAR(20) CHECK (status IN ('normal', 'abnormal', 'pending'))
);

-- ------------------------------------------------
-- Billing and Insurance
-- ------------------------------------------------

CREATE TABLE insurance_provider (
    provider_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

-- N:M relationship between patients and insurance providers
```

```sql
CREATE TABLE patient_insurance (
    patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
    provider_id INT NOT NULL REFERENCES insurance_provider(provider_id) ON
        DELETE CASCADE,
    policy_number VARCHAR(50) NOT NULL,
    PRIMARY KEY (patient_id, provider_id)
);

CREATE TABLE bill (
    bill_id SERIAL PRIMARY KEY,
    patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
    appointment_id INT UNIQUE REFERENCES appointment(appointment_id) ON
        DELETE SET NULL,
    amount NUMERIC(10, 2) NOT NULL CHECK (amount >= 0),
    issue_date DATE NOT NULL DEFAULT CURRENT_DATE,
    due_date DATE NOT NULL,
    status VARCHAR(20) NOT NULL CHECK (status IN ('unpaid', 'paid', '
        partially_paid', 'overdue'))
);

CREATE TABLE payment (
    payment_id SERIAL PRIMARY KEY,
    bill_id INT NOT NULL REFERENCES bill(bill_id) ON DELETE CASCADE,
    amount NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
    payment_date DATE NOT NULL DEFAULT CURRENT_DATE,
    payment_method VARCHAR(50) CHECK (payment_method IN ('credit_card', '
        bank_transfer', 'cash', 'insurance'))
);
```

Listing 1: Relational Schema (`schema.sql`)

3.2 Mapping Rules Applied

Key mapping decisions included:

- **N:M Relationships:** Many-to-many relationships were implemented using separate linking tables (e.g., `doctor_specialty` for Doctor–Specialty and `patient_insurance` for Patient–Insurance Provider).
- **Weak Entities:** `prescription_item` is modeled as a weak entity identified by its owning `prescription` plus a partial key (`item_number`).
- **Multivalued Attributes:** Multivalued attributes are handled via separate tables (e.g., `user_phone` and `patient_allergy`).

4 Functional Dependencies and Normalization

4.1 Functional Dependencies (FDs)

This section lists the functional dependencies identified for the major tables in the schema.

```
# Functional Dependencies (FDs) and Normal Forms

This document outlines the functional dependencies for key tables in the
    database, determines their candidate keys, and analyzes their normal
    form.
```

```
4
5   ---
6
7   ### 1. `app_user` Table
8
9   This table stores core information for all users in the system.
10
11  **Attributes:** `user_id`, `first_name`, `last_name`, `email`, `
        password_hash`, `date_of_birth`, `street`, `district`, `city`, `
        postal_code`, `country`
12
13  **Functional Dependencies:**
14  -   `{user_id} -> {first_name, last_name, email, password_hash,
        date_of_birth, street, district, city, postal_code, country}`
15      -   The primary key `user_id` uniquely determines all other attributes.
16  -   `{email} -> {user_id, first_name, last_name, password_hash,
        date_of_birth, street, district, city, postal_code, country}`
17      -   The `email` is a unique identifier for a user.
18
19  **Candidate Keys:**
20  -   `{user_id}`
21  -   `{email}`
22
23  **Normalization Analysis:**
24  -   The table is in **BCNF**.
25  -   **Reasoning:** The determinants are `{user_id}` and `{email}`. Both are
         candidate keys. Since all determinants are candidate keys, the table
        satisfies the BCNF condition.
26
27  ---
28
29  ### 2. `doctor` Table
30
31  This table is a subtype of `app_user` and stores doctor-specific
        information.
32
33  **Attributes:** `doctor_id`, `license_number`, `department_id`
34
35  **Functional Dependencies:**
36  -   `{doctor_id} -> {license_number, department_id}`
37      -   `doctor_id` is the primary key and is also a foreign key to `
            app_user(user_id)`.
38  -   `{license_number} -> {doctor_id, department_id}`
39      -   The `license_number` is a unique value assigned to each doctor.
40
41  **Candidate Keys:**
42  -   `{doctor_id}`
43  -   `{license_number}`
44
45  **Normalization Analysis:**
46  -   The table is in **BCNF**.
47  -   **Reasoning:** The determinants are `{doctor_id}` and `{license_number
        }`. Both are candidate keys. The table is in BCNF.
48
49  ---
50
51  ### 3. `appointment` Table
52
```

```
This table records appointments between patients and doctors.

**Attributes:** `appointment_id`, `patient_id`, `doctor_id`, `appointment_datetime`, `status`, `notes`

**Functional Dependencies:**
-   `{appointment_id} -> {patient_id, doctor_id, appointment_datetime, status, notes}`
    -   The primary key `appointment_id` determines all other attributes of the appointment.
-   `{patient_id, appointment_datetime} -> {appointment_id, doctor_id, status, notes}`
    -   A patient cannot have two appointments at the exact same time. This could be a candidate key if enforced with a unique constraint.
-   `{doctor_id, appointment_datetime} -> {appointment_id, patient_id, status, notes}`
    -   A doctor cannot have two appointments at the exact same time. This could also be a candidate key.

**Candidate Keys:**
-   `{appointment_id}` (Primary Key)
-   `{patient_id, appointment_datetime}` (If a business rule enforces it)
-   `{doctor_id, appointment_datetime}` (If a business rule enforces it)

**Normalization Analysis:**
-   The table is in **BCNF**.
-   **Reasoning:** Assuming `appointment_id` is the sole primary key, it is the only determinant for the main FD. Other potential FDs rely on determinants that are also candidate keys. There are no transitive dependencies (e.g., `patient_id` does not determine `doctor_id`). Therefore, the table is in BCNF.

---

### 4. `prescription_item` Table

This is a weak entity that details the medications within a single prescription.

**Attributes:** `prescription_id`, `item_number`, `medication_id`, `dosage`, `frequency`, `duration_days`

**Functional Dependencies:**
-   `{prescription_id, item_number} -> {medication_id, dosage, frequency, duration_days}`
    -   The composite key, consisting of the foreign key `prescription_id` and the partial key `item_number`, uniquely determines the details of that specific line item in the prescription.

**Candidate Keys:**
-   `{prescription_id, item_number}`

**Normalization Analysis:**
-   The table is in **BCNF**.
-   **Reasoning:** The only determinant is the composite primary key `{prescription_id, item_number}`. Since this is a superkey, the table is in BCNF. All non-key attributes (`medication_id`, `dosage`, etc.) are fully dependent on the entire primary key, satisfying 2NF, and there are
```

```
    no transitive dependencies, satisfying 3NF and BCNF.

---

### 5. `bill` Table

This table stores billing information related to appointments or other services.

**Attributes:** `bill_id`, `patient_id`, `appointment_id`, `amount`, `issue_date`, `due_date`, `status`

**Functional Dependencies:**
-   `{bill_id} -> {patient_id, appointment_id, amount, issue_date, due_date, status}`
    -   The primary key `bill_id` determines all other attributes.
-   `{appointment_id} -> {bill_id, patient_id, amount, issue_date, due_date, status}`
    -   Since each appointment generates at most one bill, `appointment_id` (which is unique) can also serve as a determinant.

**Candidate Keys:**
-   `{bill_id}`
-   `{appointment_id}` (Note: This is nullable, so it can only be a candidate key for non-null values. The `UNIQUE` constraint enforces this.)

**Normalization Analysis:**
-   The table is in **BCNF**.
-   **Reasoning:** The determinants are `{bill_id}` and `{appointment_id}`. Both are candidate keys. There are no transitive dependencies. For example, `patient_id` does not determine the `amount` or `status`. The table is well-normalized and in BCNF.
```

Listing 2: Functional Dependencies (`fd.md`)

4.2 Normalization Steps

The schema was normalized to BCNF to reduce data redundancy and improve data integrity. The process is described below.

```
# Normalization of Healthcare Data

This document explains the process of normalizing a complex, unnormalized table into a set of tables in Boyce-Codd Normal Form (BCNF).

## Unnormalized Form (UNF)

Let's start with an intentionally unnormalized table, `appointment_full_info`. This single table contains repeating groups and transitive dependencies, making it inefficient and prone to anomalies.

**`appointment_full_info`**

| appointment_id | appointment_datetime | patient_id | patient_first_name | patient_last_name | patient_dob | doctor_id | doctor_first_name | doctor_last_name | doctor_license | department_id | department_name | hospital_id | hospital_name | diagnosis_codes | prescription_meds |
```

```
| :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- |
| 101 | 2026-06-10 10:00 | 1 | John | Smith | 1985-02-10 | 6 | Emily | Davis | LIC-CARD-1122 | 1 | Cardiology | 1 | City General | "I25.10, Z00.00" | "Atorvastatin 20mg, Lisinopril 10mg" |
| 102 | 2026-06-10 11:00 | 2 | Jane | Doe | 1990-07-22 | 7 | Michael | Miller | LIC-NEUR-3344 | 2 | Neurology | 1 | City General | "G43.909" | "Sumatriptan 50mg" |

**Problems with UNF:**
- **Repeating Groups:** `diagnosis_codes` and `prescription_meds` contain multiple values in a single string. This violates 1NF.
- **Redundancy:** Patient details (name, DOB), doctor details, and department/hospital info are repeated for every appointment.
- **Anomalies:**
    - **Update Anomaly:** If Dr. Davis changes departments, we must update every record for her.
    - **Insertion Anomaly:** We can't add a new doctor until they have an appointment. We can't add a new hospital unless it has a department with a doctor who has an appointment.
    - **Deletion Anomaly:** If we delete John Smith's only appointment, we lose all information about him.

---

## First Normal Form (1NF)

**Rule:** Ensure all attributes are atomic and there are no repeating groups. Each cell should hold a single value.

We eliminate the repeating groups by creating separate tables for diagnoses and prescriptions related to an appointment.

**Decomposition:**
1. Create `appointment_diagnosis` to hold diagnoses for each appointment.
2. Create `appointment_prescription` to hold medications for each appointment.

The main table now looks like this:

**`appointment_info_1nf`**

| appointment_id (PK) | appointment_datetime | patient_id | patient_first_name | ... | hospital_name |
| :--- | :--- | :--- | :--- | :--- | :--- |
| 101 | 2026-06-10 10:00 | 1 | John | ... | City General |
| 102 | 2026-06-10 11:00 | 2 | Jane | ... | City General |

**`appointment_diagnosis`**

| appointment_id (FK) | diagnosis_code |
| :--- | :--- |
| 101 | I25.10 |
| 101 | Z00.00 |
| 102 | G43.909 |

**`appointment_prescription`**
```

```
| appointment_id (FK) | medication_name | dosage |
| :--- | :--- | :--- |
| 101 | Atorvastatin | 20mg |
| 101 | Lisinopril | 10mg |
| 102 | Sumatriptan | 50mg |

**Improvement:** The data is now atomic, making it easier to query individual diagnoses or medications. However, massive redundancy still exists.

---

## Second Normal Form (2NF)

**Rule:** Must be in 1NF, and all non-key attributes must be fully functionally dependent on the entire primary key. (This rule is most relevant for tables with composite primary keys).

Our `appointment_info_1nf` table has a single-column primary key (`appointment_id`), so it is trivially in 2NF. However, it suffers from transitive dependencies, which are addressed by 3NF. Let's identify the functional dependencies to prepare for 3NF.

**Functional Dependencies in `appointment_info_1nf`:**
- `appointment_id` -> `appointment_datetime`, `patient_id`, `doctor_id`
- `patient_id` -> `patient_first_name`, `patient_last_name`, `patient_dob` (Partial Dependency if PK was composite)
- `doctor_id` -> `doctor_first_name`, `doctor_last_name`, `doctor_license`, `department_id`
- `department_id` -> `department_name`, `hospital_id`
- `hospital_id` -> `hospital_name`

The dependencies on `patient_id`, `doctor_id`, etc., are **transitive dependencies**.

---

## Third Normal Form (3NF)

**Rule:** Must be in 2NF, and there should be no transitive dependencies. A non-key attribute cannot depend on another non-key attribute.

We decompose the table to eliminate these transitive dependencies.

**Decomposition:**
1. Create a `patient` table.
2. Create a `doctor` table.
3. Create a `department` table.
4. Create a `hospital` table.
5. The `appointment` table will only contain attributes directly related to the appointment event itself, with foreign keys to the other tables.

**Final Normalized Tables (in 3NF):**

**`hospital`**
- `hospital_id` (PK)
- `hospital_name`
- FD: `{hospital_id} -> {hospital_name}`
```

```
101
102 **`department`**
103 - `department_id` (PK)
104 - `department_name`
105 - `hospital_id` (FK)
106 - FD: `{department_id} -> {department_name, hospital_id}`
107
108 **`doctor`**
109 - `doctor_id` (PK)
110 - `doctor_first_name`, `doctor_last_name`
111 - `doctor_license` (Candidate Key)
112 - `department_id` (FK)
113 - FDs: `{doctor_id} -> {all other attributes}`, `{doctor_license} -> {all
    other attributes}`
114
115 **`patient`**
116 - `patient_id` (PK)
117 - `patient_first_name`, `patient_last_name`, `patient_dob`
118 - FD: `{patient_id} -> {all other attributes}`
119
120 **`appointment`**
121 - `appointment_id` (PK)
122 - `appointment_datetime`
123 - `patient_id` (FK)
124 - `doctor_id` (FK)
125 - FD: `{appointment_id} -> {appointment_datetime, patient_id, doctor_id}`
126
127 This structure is now in 3NF. We have eliminated the redundancy and the
    update/insertion/deletion anomalies.
128
129 ---
130
131 ## Boyce-Codd Normal Form (BCNF)
132
133 **Rule:** For every non-trivial functional dependency `X -> Y`, `X` must be
     a superkey.
134
135 BCNF is a stricter version of 3NF. A table is in BCNF if and only if every
    determinant is a candidate key.
136
137 Let's analyze our 3NF tables:
138 - **`hospital`**: The only determinant is `hospital_id`, which is the
    primary key (and thus a superkey). **In BCNF.**
139 - **`department`**: The only determinant is `department_id`, which is the
    primary key. **In BCNF.**
140 - **`patient`**: The only determinant is `patient_id`, which is the primary
     key. **In BCNF.**
141 - **`appointment`**: The only determinant is `appointment_id`, which is the
     primary key. **In BCNF.**
142 - **`doctor`**: We have two FDs:
143     1.  `{doctor_id} -> {first_name, last_name, license_number,
        department_id}`
144     2.  `{license_number} -> {doctor_id, first_name, last_name,
        department_id}`
145
146     The determinants are `{doctor_id}` and `{license_number}`. Both are
        candidate keys for the `doctor` table. Since every determinant is a
        candidate key, the `doctor` table **is in BCNF.**
```

```
147
148 **Conclusion:** The final decomposed schema (`hospital`, `department`, `
    doctor`, `patient`, `appointment`, etc.) is in BCNF, providing a robust,
     scalable, and anomaly-free design. The schema implemented in `schema.
    sql` follows this normalized structure.
```

Listing 3: Normalization Notes (`normalization.md`)

5 SQL Implementation and Advanced Queries

5.1 Data Insertion

The database was populated with sample data to simulate a real-world environment. The script below shows a few example `INSERT` statements.

```
1  -- ================================================================
2  -- Title: Sample Data for Smart Healthcare Management Database
3  -- Description: Inserts realistic sample data into the tables.
4  -- ================================================================
5
6  -- Clear existing data
7  TRUNCATE
8      patient_allergy, user_phone, payment, bill, patient_insurance,
           insurance_provider,
9      lab_result, lab_test, prescription_item, medication, prescription,
           diagnosis,
10     medical_record, appointment, doctor_specialty, specialty, admin, doctor
           ,
11     patient, department, hospital, app_user
12 RESTART IDENTITY CASCADE;
13
14 -- ------------------------------------------------
15 -- Hospitals and Departments
```

Listing 4: Sample Data Insertion (`data.sql`)

5.2 Advanced SQL Queries

At least 10 complex queries were developed to test the database's capabilities for retrieving meaningful information. These queries include joins, aggregations, and subqueries.

```
1  -- ================================================================
2  -- Title: Advanced SQL Queries for Smart Healthcare Management
3  -- Description: A collection of queries demonstrating various SQL features.
4  -- ================================================================
5
6  -- Query 1: List all scheduled appointments with patient, doctor, hospital,
       and department names.
7  -- Demonstrates: INNER JOIN on multiple tables.
8  SELECT
9      a.appointment_datetime,
10     p_user.first_name AS patient_first_name,
11     p_user.last_name AS patient_last_name,
12     d_user.first_name AS doctor_first_name,
13     d_user.last_name AS doctor_last_name,
14     h.name AS hospital_name,
15     dpt.name AS department_name
```

```
FROM appointment a
JOIN patient p ON a.patient_id = p.patient_id
JOIN app_user p_user ON p.patient_id = p_user.user_id
JOIN doctor doc ON a.doctor_id = doc.doctor_id
JOIN app_user d_user ON doc.doctor_id = d_user.user_id
JOIN department dpt ON doc.department_id = dpt.department_id
JOIN hospital h ON dpt.hospital_id = h.hospital_id
WHERE a.status = 'scheduled'
ORDER BY a.appointment_datetime;


-- Query 2: Find doctors who have more than 2 appointments (scheduled or
    completed).
-- Demonstrates: GROUP BY, HAVING, COUNT, and subquery in WHERE IN.
SELECT
    d_user.first_name,
    d_user.last_name,
    COUNT(a.appointment_id) AS appointment_count
FROM doctor doc
JOIN app_user d_user ON doc.doctor_id = d_user.user_id
JOIN appointment a ON doc.doctor_id = a.doctor_id
WHERE a.status IN ('scheduled', 'completed')
GROUP BY doc.doctor_id, d_user.first_name, d_user.last_name
HAVING COUNT(a.appointment_id) > 2
ORDER BY appointment_count DESC;


-- Query 3: List patients who have no appointments at all.
-- Demonstrates: LEFT JOIN and IS NULL to find non-matching records.
SELECT
    p_user.user_id,
    p_user.first_name,
    p_user.last_name
FROM patient p
JOIN app_user p_user ON p.patient_id = p_user.user_id
LEFT JOIN appointment a ON p.patient_id = a.patient_id
WHERE a.appointment_id IS NULL;


-- Query 4: Find the most common diagnosis based on ICD-10 code.
-- Demonstrates: GROUP BY, COUNT, ORDER BY, LIMIT.
SELECT
    icd10_code,
    description,
    COUNT(*) AS diagnosis_count
FROM diagnosis
WHERE icd10_code IS NOT NULL
GROUP BY icd10_code, description
ORDER BY diagnosis_count DESC
LIMIT 5;


-- Query 5: Calculate the total bill amount and total amount paid per
    patient.
-- Demonstrates: Subqueries in SELECT, LEFT JOIN, COALESCE, SUM, GROUP BY.
SELECT
    p_user.first_name,
    p_user.last_name,
```

```
    COALESCE(SUM(b.amount), 0) AS total_billed,
    (SELECT COALESCE(SUM(p.amount), 0)
     FROM payment p
     JOIN bill b2 ON p.bill_id = b2.bill_id
     WHERE b2.patient_id = p_user.user_id) AS total_paid
FROM patient pat
JOIN app_user p_user ON pat.patient_id = p_user.user_id
LEFT JOIN bill b ON pat.patient_id = b.patient_id
GROUP BY p_user.user_id, p_user.first_name, p_user.last_name
ORDER BY total_billed DESC;


-- Query 6: List all unpaid or partially paid bills that are past their due
    date (overdue).
-- Demonstrates: Filtering by date, multiple conditions in WHERE.
SELECT
    b.bill_id,
    p_user.first_name,
    p_user.last_name,
    b.amount,
    b.due_date,
    b.status
FROM bill b
JOIN app_user p_user ON b.patient_id = p_user.user_id
WHERE b.status IN ('unpaid', 'partially_paid')
  AND b.due_date < CURRENT_DATE
ORDER BY b.due_date;


-- Query 7: Find medications that have been prescribed more than 3 times.
-- Demonstrates: Nested query, GROUP BY, HAVING.
SELECT
    m.name,
    m.description,
    pi_counts.prescription_count
FROM medication m
JOIN (
    SELECT
        medication_id,
        COUNT(*) AS prescription_count
    FROM prescription_item
    GROUP BY medication_id
    HAVING COUNT(*) >= 3
) AS pi_counts ON m.medication_id = pi_counts.medication_id
ORDER BY pi_counts.prescription_count DESC;


-- Query 8: Show all lab results for a specific patient (John Smith,
   user_id=1).
-- Demonstrates: JOIN, filtering by a specific ID.
SELECT
    lr.result_date,
    lt.name AS test_name,
    lr.result_value,
    lr.status
FROM lab_result lr
JOIN lab_test lt ON lr.test_id = lt.test_id
WHERE lr.patient_id = 1
```

```sql
ORDER BY lr.result_date DESC;


-- Query 9: Find doctors in the 'Cardiology' department at 'City General Hospital' and list their specialties.
-- Demonstrates: Multiple JOINs, filtering by text fields.
SELECT
    d_user.first_name,
    d_user.last_name,
    s.name AS specialty_name
FROM doctor doc
JOIN app_user d_user ON doc.doctor_id = d_user.user_id
JOIN department dpt ON doc.department_id = dpt.department_id
JOIN hospital h ON dpt.hospital_id = h.hospital_id
JOIN doctor_specialty ds ON doc.doctor_id = ds.doctor_id
JOIN specialty s ON ds.specialty_id = s.specialty_id
WHERE h.name = 'City General Hospital'
  AND dpt.name = 'Cardiology';


-- Query 10: Find patients who have appointments but have no lab results.
-- Demonstrates: NOT EXISTS, correlated subquery.
SELECT DISTINCT
    p_user.first_name,
    p_user.last_name
FROM patient pat
JOIN app_user p_user ON pat.patient_id = p_user.user_id
JOIN appointment a ON pat.patient_id = a.patient_id
WHERE NOT EXISTS (
    SELECT 1
    FROM lab_result lr
    WHERE lr.patient_id = pat.patient_id
);

-- Query 11: Find the doctor with the highest number of distinct patients.
-- Demonstrates: COUNT(DISTINCT), GROUP BY, ORDER BY, LIMIT.
SELECT
    d_user.first_name,
    d_user.last_name,
    COUNT(DISTINCT a.patient_id) AS distinct_patient_count
FROM appointment a
JOIN doctor doc ON a.doctor_id = doc.doctor_id
JOIN app_user d_user ON doc.doctor_id = d_user.user_id
GROUP BY doc.doctor_id, d_user.first_name, d_user.last_name
ORDER BY distinct_patient_count DESC
LIMIT 1;


-- Query 12: Find the average bill amount by department for completed appointments.
-- Demonstrates: AVG, GROUP BY, JOIN across multiple tables.
SELECT
    dpt.name AS department_name,
    h.name AS hospital_name,
    AVG(b.amount) AS average_bill_amount
FROM bill b
JOIN appointment a ON b.appointment_id = a.appointment_id
JOIN doctor doc ON a.doctor_id = doc.doctor_id
```

```sql
JOIN department dpt ON doc.department_id = dpt.department_id
JOIN hospital h ON dpt.hospital_id = h.hospital_id
WHERE a.status = 'completed'
GROUP BY dpt.name, h.name
ORDER BY average_bill_amount DESC;
```

Listing 5: Advanced Queries (`queries.sql`)

5.2.1 Query Outputs

For each query in `queries.sql`, you can run it and paste the output here (e.g., inside a `verbatim` environment or as a formatted table).

Query 1: List scheduled appointments with patient/doctor/hospital/department.

```
-- Output of Query 1 here...
```

Query 3: List patients with no appointments.

```
-- Output of Query 3 here...
```

6 Relational Algebra & Calculus

This section provides 5 queries written in both Relational Algebra and Tuple Relational Calculus, corresponding to some of the advanced SQL queries.

```
# Relational Algebra and Tuple Relational Calculus Examples

This document provides 5 query examples written in natural language,
    relational algebra, and tuple relational calculus.

---

### Query 1: Find the names of all patients who have a scheduled
    appointment.

-   **Relational Algebra:**
    \Pi_{first_name, last_name}(
        (app_user \bowtie_{app_user.user_id = patient.patient_id} patient)
        \bowtie_{patient.patient_id = appointment.patient_id}
        (\sigma_{status='scheduled'}(appointment))
    )

-   **Tuple Relational Calculus:**
    `{ t | \exists u \in app_user, \exists p \in patient, \exists a \in
        appointment (`
    `u.user_id = p.patient_id \wedge p.patient_id = a.patient_id \wedge a.
        status = 'scheduled' \wedge`
    `t.first_name = u.first_name \wedge t.last_name = u.last_name) }`

---

### Query 2: Find the names of all doctors working in the 'Cardiology'
    department.

-   **Relational Algebra:**
    \Pi_{first_name, last_name}(
        (app_user \bowtie_{app_user.user_id = doctor.doctor_id} doctor)
```

```
        \bowtie_{doctor.department_id = department.department_id}
        (\sigma_{name='Cardiology'}(department))
    )

-   **Tuple Relational Calculus:**
    `{ t | \exists u \in app_user, \exists d \in doctor, \exists dept \in department (`
    `u.user_id = d.doctor_id \wedge d.department_id = dept.department_id \wedge dept.name = 'Cardiology' \wedge`
    `t.first_name = u.first_name \wedge t.last_name = u.last_name) }`

---

### Query 3: Find the amount and due date of all unpaid bills.

-   **Relational Algebra:**
    \Pi_{amount, due_date}(\sigma_{status='unpaid'}(bill))

-   **Tuple Relational Calculus:**
    `{ t | \exists b \in bill (b.status = 'unpaid' \wedge t.amount = b.amount \wedge t.due_date = b.due_date) }`

---

### Query 4: Find the names of patients who have at least one 'abnormal' lab result.

-   **Relational Algebra:**
    \Pi_{first_name, last_name}(
        (app_user \bowtie_{app_user.user_id = patient.patient_id} patient)
        \bowtie_{patient.patient_id = lab_result.patient_id}
        (\sigma_{status='abnormal'}(lab_result))
    )

-   **Tuple Relational Calculus:**
    `{ t | \exists u \in app_user, \exists p \in patient, \exists lr \in lab_result (`
    `u.user_id = p.patient_id \wedge p.patient_id = lr.patient_id \wedge lr.status = 'abnormal' \wedge`
    `t.first_name = u.first_name \wedge t.last_name = u.last_name) }`

---

### Query 5: Find the names of all medications prescribed to the patient with `patient_id = 1`.

-   **Relational Algebra:**
    \Pi_{name}(
        medication \bowtie_{medication.medication_id = prescription_item.medication_id}
        prescription_item \bowtie_{prescription_item.prescription_id = prescription.prescription_id}
        prescription \bowtie_{prescription.record_id = medical_record.record_id}
        (\sigma_{patient_id=1}(medical_record))
    )

-   **Tuple Relational Calculus:**
```

```
    `{ t | \exists m \in medication, \exists pi \in prescription_item, \
       exists p \in prescription, \exists mr \in medical_record (`
    `m.medication_id = pi.medication_id \wedge pi.prescription_id = p.
       prescription_id \wedge p.record_id = mr.record_id \wedge mr.
       patient_id = 1 \wedge`
    `t.name = m.name) }`
```

Listing 6: Relational Algebra and Tuple Relational Calculus (`relational_algebra_and_calculus.md`)

7 Indexing and File Structures

7.1 Index Implementation

To optimize query performance, several indexes were created. The choices include B+ Tree indexes for range queries and Hash indexes for exact lookups where applicable.

```sql
-- ================================================================
-- Title: Indexing and Performance Analysis
-- Description: Demonstrates index creation and performance comparison.
-- ================================================================

-- Clean up any existing indexes to ensure a clean run
DROP INDEX IF EXISTS idx_appointment_patient_id;
DROP INDEX IF EXISTS idx_appointment_doctor_datetime;
DROP INDEX IF EXISTS idx_user_email_hash;
DROP INDEX IF EXISTS idx_bill_patient_status;
DROP INDEX IF EXISTS idx_lab_result_patient_date;

-- ------------------------------------------------
-- Example 1: B+ Tree Index on a Foreign Key
-- ------------------------------------------------
-- Query: Find all appointments for a specific patient.
-- This is a very common operation in a healthcare system.

-- Before index:
EXPLAIN ANALYZE
SELECT * FROM appointment WHERE patient_id = 1;

-- Result (example): Seq Scan on appointment (cost=0.00..45.50 rows=5 width
   =33) (actual time=...)
-- The database has to scan the entire 'appointment' table.

-- Create a B+ Tree index (the default type in PostgreSQL)
CREATE INDEX idx_appointment_patient_id ON appointment(patient_id);

-- After index:
EXPLAIN ANALYZE
SELECT * FROM appointment WHERE patient_id = 1;

-- Result (example): Index Scan using idx_appointment_patient_id on
   appointment (cost=0.28..8.29 rows=5 width=33) (actual time=...)
-- The database now uses the index for a much faster lookup.

-- ------------------------------------------------
-- Example 2: Composite B+ Tree Index
-- ------------------------------------------------
```

```sql
-- Query: Find all appointments for a doctor within a specific date range.
-- This is useful for a doctor's calendar view.

-- Before index:
EXPLAIN ANALYZE
SELECT appointment_id, appointment_datetime, status
FROM appointment
WHERE doctor_id = 6 AND appointment_datetime >= '2026-06-01' AND
    appointment_datetime < '2026-07-01';

-- Result (example): Seq Scan on appointment... (filters on both doctor_id
    and datetime)

-- Create a composite index on doctor_id and appointment_datetime
CREATE INDEX idx_appointment_doctor_datetime ON appointment(doctor_id,
    appointment_datetime);

-- After index:
EXPLAIN ANALYZE
SELECT appointment_id, appointment_datetime, status
FROM appointment
WHERE doctor_id = 6 AND appointment_datetime >= '2026-06-01' AND
    appointment_datetime < '2026-07-01';

-- Result (example): Index Scan using idx_appointment_doctor_datetime...
-- The index allows efficient lookup of the doctor and then scanning the
    relevant date range within that doctor's appointments.

-- ------------------------------------------------
-- Example 3: Hash Index for Exact Lookups
-- ------------------------------------------------
-- Query: Find a user by their exact email address.
-- This is common during login or user registration checks. Hash indexes
    are optimized for equality checks.

-- Before index (assuming only the UNIQUE constraint's B-Tree index exists)
    :
EXPLAIN ANALYZE
SELECT user_id, first_name, email FROM app_user WHERE email = 'jane.
    doe@example.com';

-- Result (example): Index Scan using app_user_email_key... (B-Tree)

-- Create a HASH index. Note: B-Tree is often good enough, but this
    demonstrates the syntax.
CREATE INDEX idx_user_email_hash ON app_user USING HASH (email);

-- After index:
EXPLAIN ANALYZE
SELECT user_id, first_name, email FROM app_user WHERE email = 'jane.
    doe@example.com';

-- Result (example): Bitmap Heap Scan on app_user... -> Bitmap Index Scan
    on idx_user_email_hash...
-- The planner might choose the hash index for this equality-only query,
    which can be more memory-efficient and faster for very large tables.

-- ------------------------------------------------
```

```
-- Example 4: Composite Index for Filtering and Sorting
-- ------------------------------------------------
-- Query: Find unpaid bills for a patient, ordered by due date.
-- This helps in displaying a patient's outstanding bills in a logical order.

-- Before index:
EXPLAIN ANALYZE
SELECT bill_id, amount, due_date
FROM bill
WHERE patient_id = 3 AND status = 'partially_paid'
ORDER BY due_date;

-- Result (example): Seq Scan on bill... followed by a Sort operation. Sorting can be expensive.

-- Create a composite index to cover filtering and ordering
CREATE INDEX idx_bill_patient_status ON bill(patient_id, status, due_date);

-- After index:
EXPLAIN ANALYZE
SELECT bill_id, amount, due_date
FROM bill
WHERE patient_id = 3 AND status = 'partially_paid'
ORDER BY due_date;

-- Result (example): Index Scan using idx_bill_patient_status...
-- The database can use the index to find the matching rows and retrieve them in the pre-sorted order of 'due_date', avoiding a separate Sort step.

-- ------------------------------------------------
-- Example 5: Index on Lab Results
-- ------------------------------------------------
-- Query: Retrieve a patient's lab history, sorted by date.
-- A common query for reviewing a patient's medical history.

-- Before index:
EXPLAIN ANALYZE
SELECT result_id, test_id, result_date, result_value
FROM lab_result
WHERE patient_id = 1
ORDER BY result_date DESC;

-- Result (example): Seq Scan on lab_result... followed by a Sort.

-- Create a composite index
CREATE INDEX idx_lab_result_patient_date ON lab_result(patient_id, result_date DESC);

-- After index:
EXPLAIN ANALYZE
SELECT result_id, test_id, result_date, result_value
FROM lab_result
WHERE patient_id = 1
ORDER BY result_date DESC;

-- Result (example): Index Scan using idx_lab_result_patient_date...
```

```
138 -- The index provides fast access to the patient's results and delivers
    them in the correct descending order by date.
```

Listing 7: Index Creation (`indexing.sql`)

7.2 Performance Analysis

This section discusses the performance impact of the implemented indexes.

7.2.1 Indexed vs. Non-Indexed Query Performance

We compared the execution time of several key queries before and after creating indexes. For example, finding all appointments for a specific patient, retrieving a doctor's appointments within a date range, or listing a patient's lab history sorted by date.

Example Query: Find all appointments for a specific patient.

- **Without Index:** The database may perform a sequential scan on the `appointment` table. Execution time: XX ms.
- **With B+ Tree Index on `appointment(patient_id)`:** The database can use an index scan to quickly locate matching rows. Execution time: YY ms.

The results clearly demonstrate that using indexes significantly reduces query execution time, especially for large datasets. The choice between B+ Tree and Hash index depends on the query type, with B+ Trees being more versatile for both range and point queries.